

Python编程实现粒子群算法(PSO)详解

主要介绍了Python编程实现粒子群算法(PSO)详解，涉及粒子群算法的原理，过程，以及实现代码示例，具有一定参考价值，需要的朋友可以了解下。

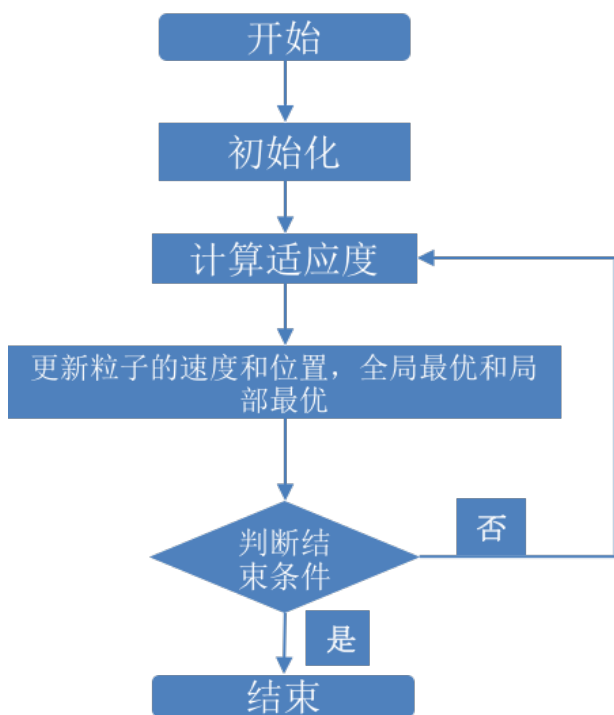
1 原理

粒子群算法是群智能一种，是基于对鸟群觅食行为的研究和模拟而来的。假设在鸟群觅食范围，只在一个地方有食物，所有鸟儿看不到食物（不知道食物的具体位置），但是能闻到食物的味道（能知道食物距离自己位置）。最好的策略就是结合自己的经验在距离鸟群中距离食物最近的区域搜索。

利用粒子群算法解决实际问题本质上就是利用粒子群算法求解函数的最值。因此需要事先把实际问题抽象为一个数学函数，称之为适应度函数。在粒子群算法中，每只鸟都可以看成是问题的一个解，这里我们通常把鸟称之为粒子，每个粒子都拥有：

位置，可以理解函数的自变量的值；
经验，也即是自身经历过的距离食物最近的位置；
速度，可以理解为自变量的变化值；
适应度，距离食物的位置，也就是函数值。

粒子群算法的过程



PSO流程图

初始化。包括根据给定的粒子个数，初始化粒子，包括初始化一下的值：

位置：解空间内的随机值；
经验：与初始位置相等；
速度：0；
适应度：根据位置，带入适应度函数，得到适应度值。

更新。包括两部分：

粒子自身信息：包括根据下面的公式更新粒子的速度、位置，根据适应度函数更新适应度，然后和用更新后的适应度和自身经验进行比较，如果新的适应度由于经验的适应度，就利用当前位置更新经验；

$$v_i(t) = w * v_{i-1}(t-1) + c_1 * r_1(x_{pbest}(t-1) - x_i(t-1)) + c_2 * r_2(x_{gbest}(t-1) - x_i(t-1))$$

速度更新公式

$$x_i(t) = x_i(t-1) + v_i(t)$$

位置更新公式

上面公式中：i表示粒子编号；t表示时刻，反映在迭代次数上；w是惯性权重，一般设置在0.4左右；c表示学习因子，一般都取值为2；Xpbest表示的是粒子i的经验，也即是粒子i所到过最佳位置；Xgbest代表的是全局最优粒子的位置；r是0到1之间的随机值。

种群信息：把当前适应度和全局最优位置的适应度进行比较，如果当前适应度优于全局最优的适应度，那么久用当前粒子替换群居最优。

判断结束条件。结束条件包括最大迭代次数和适应度的阈值。

2 代码

实验环境为python 2.7.11。

这个代码最初是用于求解一维最大熵分割图像问题的，因此是求解函数最大值，如果要求解最小值，把代码中的大于号全部改成小于号就可以了。

首先需要解决的是粒子的存储，我第一反应是利用结构体来存储，但是python并没有相应的数据结构，所以我选择用一个类来表示粒子结构，该类的一个对象就是一个粒子，上代码：

```
class bird:
    """
    speed:速度
    position:位置
    fit:适应度
    lbestposition:经历的最佳位置
    lbestfit:经历的最佳的适应度值
    """

    def __init__(self, speed, position, fit, lBestPosition, lBestFit):
        self.speed = speed
        self.position = position
        self.fit = fit
        self.lBestFit = lBestPosition
        self.lBestPosition = lPestFit
```

接下来就是粒子群算法的主干部分，用一个类来封装，代码：

```
import random

class PSO:
    """
    fitFunc:适应度函数
    birdNum:种群规模
    w:惯性权重
    c1,c2:个体学习因子，社会学习因子
    solutionSpace:解空间，列表类型：[最小值，最大值]
    """

    def __init__(self, fitFunc, birdNum, w, c1, c2, solutionSpace):
        self.fitFunc = fitFunc
        self.w = w
        self.c1 = c1
        self.c2 = c2
        self.birds, self.best = self.initbirds(birdNum, solutionSpace)

    def initbirds(self, size, solutionSpace):
        birds = []
        for i in range(size):
            position = random.uniform(solutionSpace[0], solutionSpace[1])
            speed = 0
            fit = self.fitFunc(position)
            birds.append(bird(speed, position, fit, position, fit))
            best = birds[0]
        for bird in birds:
            if bird.fit > best.fit:
                best = bird
        return birds,best

    def updateBirds(self):
        for bird in self.birds:
            # 更新速度
            bird.speed = self.w * bird.speed + self.c1 * random.random() * (bird.lBestPosition - bird.position) + self.c2 * random.random() * (self.best.position - bird.position)
            # 更新位置
            bird.position = bird.position + bird.speed
            # 跟新适应度
            bird.fit = self.fitFunc(bird.position)
            # 查看是否需要更新经验最优
            if bird.fit > bird.lBestFit:
                bird.lBestFit = bird.fit
                bird.lBestPosition = bird.position

    def solve(self, maxIter):
        # 只考虑了最大迭代次数，如需考虑阈值，添加判断语句就好
        for i in range(maxIter):
            # 更新粒子
            self.updateBirds()
            for bird in self.birds:
                # 查看是否需要更新全局最优
                if bird.fit > self.best.fit:
```

```
self.best = bird
```

有了以上代码，只需要自定义适应度函数fitFunc就可以进行求解，但是需要注意的是只适用于求解 一维问题。

总结

以上就是本文关于Python编程实现粒子群算法(PSO)详解的全部内容，希望对大家有所帮助。感兴趣的朋友可以继续参阅本站：Python算法输出1-9数组形成的结果为100的所有运算式、Python内存管理方式和垃圾回收算法解析、Python随机生成均匀分布在单位圆内的点代码示例等，有什么问题可以随时留言，小编会及时回复大家的。感谢朋友们对本站的支持！